Creating Class and Object

Lecture 7

Based on Slides of Dr. Norazah Yusof

Classes

A Java class member may have:

- **instance variables** to define data fields/attributes and

- **instance methods** to define behaviors.

Additionally, a class provides a special type of methods, known as **constructors**, which are invoked to construct objects from the class.

UML Class Diagram for Employee

Employee
- empnum: int - empname: String
+ setEmpNum(int): void + getEmpNum(): int + setEmpName(String): void + getEmpName(): String

Class name

Attributes / instance variables that define the object's state; can hold numbers, characters, strings, other objects

Actions that an object of this class can take (behaviors)

Data Fields

- Variables declared within class
  - But outside of any method
- Called instance variables (local)
  - When not `static` (global)
- Each class will have own copy of each non-static data field
- `private` access for fields
  - No other classes can access field's values
  - Only methods of same class allowed to use `private` variables

Data Fields (cont'd)

You name it!

private [static] [final] *datatype* *name*;

Usually private

May be present: means the field is shared by all objects in the class

May be present: means the field is a constant

primitive: **int**, **double**, etc., or an object: **String**, **Image**, **Foot**

```
private int empnum;
```

Creating Instance Methods in a Class

- **Classes contain methods**
- **Non-static method**
  - **Instance methods**
  - **“Belong” to objects**
- **Typically declare non-static instance variables**
- **`static` class variables**
  - **Are not instance variables**

7

Method Definitions

- **Method header form:**

  ***modifier returnType methodName* ([*parameterList*])**

- **Example:**

```
public double getPrice()
public void setPrice(double aPrice)
```

- **Method names usually sound like verbs.**
- **The name of a method that returns the value of a field often starts with get:**

  `getWidth, getX`

- **The name of a method that sets the value of a field often starts with set:**

  `setLocation, setText`

Understanding Data Hiding

- Data hiding using encapsulation
  - Data fields usually **`private`**
  - Client application accesses them only through **`public`** interfaces
- Set method
  - Controls data values used to set variable
- Get method
  - Controls how value retrieved

Accessor and Mutator Methods

- **The methods that retrieve the data of fields are called *accessors*.**
- **The methods that modify the data of fields are called *mutators*.**

Accessors and Mutators

- **For the rectangle example, the accessors and mutators are:**
  - **getLength: Returns the value of the length field.**

    ```
    public double getLength() { }
    ```
  - **getWidth : Returns the value of the width field.**

    ```
    public double getWidth() { }
    ```
  - **setLength: Sets the value of the length field.**

    ```
    public void setLength(double len) { }
    ```
  - **setWidth : Sets the value of the width field.**

    ```
    public void setLength(double w) { }
    ```

Class Definition

Employee
- empnum: int - empname: String
+ setEmpNum(int): void + getEmpNum(): int + setEmpName(String): void + getEmpName(): String

UML class diagram

```
public class Employee {
  private int empnum;
  private String empname;

  public void setEmpNum(int num) {
   empnum = num;
   }

  public int getEmpNum() {
    return empnum;
   }

 public void setEmpName(Strig name) {
    empname = name;
  }

 public String getEmpName() {
    return empname;
  }
}
```

Declaring Object Reference Variable and Create Object

- **To refer an object, assign the object to a reference variable.**
- **To declare a reference variable, use the syntax:**

```
ClassName objectRefVar;
```

- **To create object:**

```
objectRefVar = new ClassName();
```

Example:

```
Employee emp1;
emp1 = new Employee();
```

Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Assign object reference

Create an object

Example:

```
Employee emp1 = new Employee();
```

Accessing Objects

- **After object instantiated, methods accessed using:**
  - **Object's identifier**
  - **Dot**
  - **Data/Method call**
- **Referencing the object's data:**

```
objectRefVar.data
```

- **Example:**

```
emp1.empnum
```

- **Invoking the object's method:**

```
objectRefVar.methodName(arguments)
```

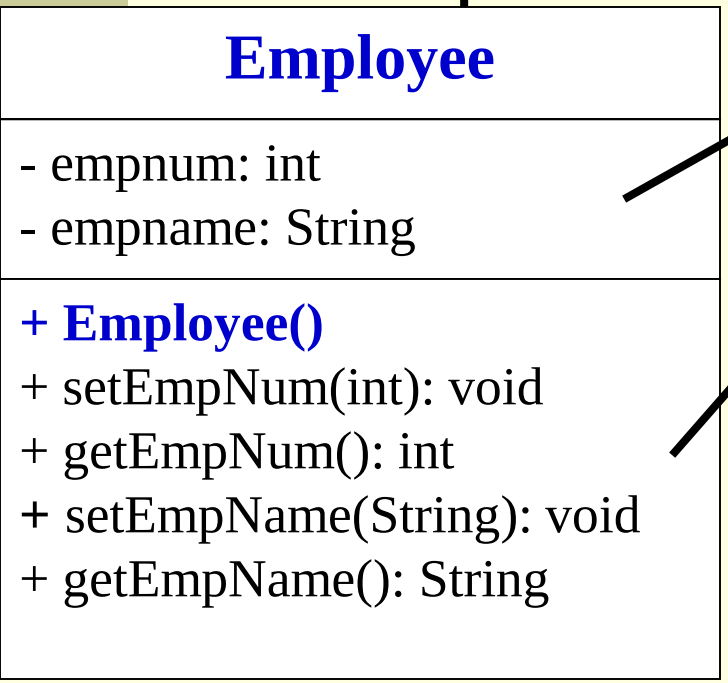
- **Example:**

```
emp1.getEmpNum()
```

Constructors

- **Special kind of methods for creating objects of the class.**
- **A class may be declared without constructors.**
- **If a programmer does not define any constructors, Java will automatically provides one default no-args constructor, which allocates memory and sets fields to the default values**
- **Name of constructor always same as name of class.**
- **Default constructors do not have a return type (not even void) and they do not return a value.**
- **Example:** `public Employee() {  }`

Class Definition with default constructor



UML class diagram

```
public class Employee {
  private int empnum;
  private String empname;

  public Employee() {
    System.out.println ("Start Employee");
  }

  public void setEmpNum(int num) {
   empnum = num;
   }
  public int getEmpNum() {
    return empnum;
   }
 public void setEmpName(String name) {
    empname = name;
  }
 public String getEmpName() {
    return empname;
  }
}
```

Constructors

- Constructors may take parameters.
  - If a constructor has one parameter, we may call it as **one-argument constructor**.
- If a class has **more than one constructor**, they must have different numbers and/or types of parameters.
  - This is called **constructor overloading**

Creating Object == invoking Constructors

- **Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.**
- **Format:**

```
new ClassName();
```

Example:

```
new Employee();
new Employee(123, "Ahmad");
```

Constructors (cont'd)

```
public class Fraction
{
   private int num, denom;

   public Fraction ( )
   {
      num = 0;
      denom = 1;
   }

   public Fraction (int n)
   {
      num = n;
      denom = 1;
   }
```

"No-args" constructor

Continued ⇨

```
   public Fraction (int n, int d)
   {
      num = n;
      denom = d;
      reduce ();
   }

   public Fraction (Fraction other)
   {
      num = other.num;
      denom = other.denom;
   }
   ...

}
```

Copy constructor

Constructors (cont'd)

- A nasty bug:

```
public class MyWindow
                    extends JFrame
{
   ...
   // Constructor:
   public void MyWindow ( )
   {
      ...
   }
   ...
```

Compiles fine, but the compiler thinks this is a <u>method</u> and uses **MyWindow**'s default no-args constructor instead.

Constructors (cont'd)

- Constructors of a class can call each other using the keyword **this** — a good way to avoid duplicating code:

```
public class Fraction
{
   ...

   public Fraction (int n)
   {
      this (n, 1);
   }
   ...
```

```
   ...
   public Fraction (int p, int q)
   {
      num = p;
      denom = q;
      reduce ();
   }
   ...
```

Organizing Classes

- Place **data fields** in logical order
  - At beginning of class
  - Fields listed vertically
- May place data fields and methods in any order within a class
  - Common to list all data fields first
  - Can see names and data types before reading methods that use data fields

Employee.java

import ...

import statements

public class Employee

Class header

{

- Fields

Attributes / variables that define the object's state; can hold numbers, characters, strings, other objects

- Constructors

Procedures for constructing a new object of this class and initializing its fields

- Methods

Actions that an object of this class can take (behaviors)

}

Lab Exercise 2

- **Exercise 1 - 6 (page 17 - 27)**